

Here is your updated, comprehensive **Cloud Security Engineering Master Checklist**. This checklist tracks your entire journey, reflecting your shift from manual configuration to active monitoring and automation, completely customized for **dev-custom-vpc**.



Current Progress Report

- **Completed Modules:** 6 / 8 (75% of Core Framework Done)
- **Current Status:** Ready to transition from manual cloud administration to enterprise-level Automation & DevSecOps.



COMPLETED WORKS (Your Verified Skillset)

Phase 1: The Secure Foundation (Identity & Cost)

-  **Root Account Security:** Secured the root user account with Multi-Factor Authentication (MFA).
-  **Identity Management (IAM):** Provisioned a dedicated admin user (study-admin) to enforce the Principle of Least Privilege.
-  **FinOps Defense:** Configured a "Zero-Spend" AWS Budget with alarms to prevent unexpected charges.
-  **Multi-Cloud Awareness:** Established parallel billing monitors within Google Cloud Platform (GCP).

Phase 2: The Engineering Lab (Local Infrastructure)

-  **Virtualization:** Configured hypervisors (VirtualBox) to host sandboxed environments.
-  **Attacker Machine:** Deployed **Kali Linux** for future penetration testing modules.
-  **Builder Machine:** Provisioned a **Debian 13** workstation as your primary local development environment.
-  **DevSecOps Toolchain:** Installed Docker, Ansible, and Cloud CLIs (AWS, Azure, GCP).
-  **Secure Version Control:** Interfaced your local lab environment with **GitHub** using SSH key authentication.

Phase 3: Cloud Primitives (Compute & Storage)

-  **Compute Provisioning:** Launched Linux EC2 instances, managed .pem key pairs, and locked down local file permissions (chmod 400).
-  **Micro-Firewalls:** Implemented Security Groups to whitelist essential protocols (SSH/Port 22, HTTP/Port 80).
-  **Web Administration:** Installed and monitored NGINX web servers using core Linux networking utilities (ss -tulnp).
-  **Object Storage Security:** Created private S3 buckets, verified "Block Public Access" configurations, and generated temporary **Pre-signed URLs** for secure asset delivery.

Phase 4: Network Architecture (The "Fortress")

-  **Custom Network Design:** Deleted the default AWS network topology and built **dev-custom-vpc** from scratch.

-  **Network Segmentation:** Created a public subnet ("The Lobby") and a private subnet ("The Vault").
-  **Edge Routing:** Implemented Internet Gateways (IGW) and Route Tables to govern edge boundaries.
-  **Isolation Verification:** Confirmed public accessibility for frontend servers while verifying backend servers remained hidden.

Phase 5: Advanced Connectivity (Secure Bridges)

-  **Inbound Gateways (Bastion Host):** Deployed a hardened jump box in the public subnet to proxy SSH connections to the private tier.
-  **Security Group Chaining:** Configured the private database firewall to accept traffic *exclusively* from the Bastion's Security Group ID.
-  **Outbound Translation (NAT Gateway):** Provisioned a NAT Gateway with an Elastic IP to give your isolated servers a secure, one-way mirror path to the internet.
-  **Package Deployment:** Validated outbound traffic by securely passing through the NAT Gateway to download, install, and run NGINX on an unaddressed private instance.

Phase 6: Cloud Defense & Monitoring (Active Telemetry)

-  **Stateless Firewalls:** Configured Network ACLs (NACLs) to regulate traffic crossing the public subnet boundary, managing ephemeral port rules.
-  **Network Auditing:** Enabled VPC Flow Logs to stream raw accept/reject packet data directly into CloudWatch Log Groups.
-  **Performance Load Testing:** Installed and executed the Linux stress utility on Amazon Linux 2023 to force extreme hardware utilization.
-  **Automated Incident Alerting:** Linked CloudWatch CPU Utilization metrics to Simple Notification Service (SNS) to trigger live email notifications when systems breach thresholds.
-  **FinOps Tear-Down:** Safely decommissioned all experimental infrastructure to maintain a strict zero-dollar operational overhead.



REMAINING WORKS (Your Future Roadmap)

This is the remaining 25% of the engineering framework designed to bridge your manual cloud foundations with professional DevSecOps engineering practices.

Phase 7: Automation & Infrastructure as Code (IaC) — NEXT TASK

-  **Terraform Installation:** Set up the HashiCorp repository on your **Debian 13** workstation.
-  **Provider Initialization:** Authenticate Terraform with your AWS CLI study-admin environment variables.
-  **Declarative Networking:** Write declarative .tf configuration files to code the entire network (dev-custom-vpc, subnets, route tables, and gateways).
-  **State Management:** Learn how Terraform keeps track of actual cloud assets using state files.

-  **The "One-Command" Test:** Build and destroy your entire architecture instantly using terraform apply and terraform destroy.

Phase 8: Advanced Application Defense & Inspection

-  **WAF (Web Application Firewall):** Deploy rules to shield web applications from application-layer threats like SQL Injection (SQLi) and Cross-Site Scripting (XSS).
-  **Data Encryption (KMS):** Implement Key Management Service to manage envelope encryption over EBS volumes and S3 buckets.
-  **Threat Detection (GuardDuty):** Enable intelligent threat detection to analyze continuous account activity for unusual anomalies (e.g., unauthorized API requests or rogue compute behavior).

Next Immediate Action Item

We are picking up exactly where your cleanup left off by entering **Phase 7**.

Your next objective is to log into your **Debian 13 terminal** so we can install **Terraform** and begin coding your infrastructure. Let me know when you are at your terminal and ready to begin!